

WYKŁAD 6

Programowanie obiektowe I: klasy i dziedziczenie

Kurs języka Python — semestr 2025/2026

Wprowadzenie do programowania II | Informatyka Stosowana

Akademia Nauk Stosowanych w Nowym Sączu

Tematy

- | | | |
|---|--|--|
| 1 | Klasy — fundamenty | <code>class</code> , <code>__init__</code> , <code>self</code> , atrybuty |
| 2 | Metody | instancji, <code>@classmethod</code> , <code>@staticmethod</code> |
| 3 | <code>__str__</code> i <code>__repr__</code> | reprezentacja tekstowa obiektu |
| 4 | Properties i enkapsulacja | <code>@property</code> , <code>_</code> , <code>__</code> , <code>__slots__</code> |
| 5 | Dziedziczenie | <code>super()</code> , <code>override</code> , <code>isinstance()</code> |
| 6 | Przeciążanie operatorów | dunder methods, <code>__add__</code> , <code>__eq__</code> ... |
| 7 | Przykłady i ciekawostki | <code>Wektor2D</code> , <code>Singleton</code> , <code>total_ordering</code> |

Klasy - fundamenty

class, __init__, self, atrybuty instancji i klasy

Czym jest klasa?

Klasa = przepis (szablon) na tworzenie obiektów

Klasa definiuje:

atrybuty

→ dane (stan obiektu)

metody

→ zachowania (co obiekt potrafi)

```
class Pies:
    gatunek = 'Canis familiaris'

    def __init__(self, imie, wiek):
        self.imie = imie
        self.wiek = wiek

    def szczekaj(self):
        return f'{self.imie}: Hau!'
```

Analogia:

Klasa to forma do ciastek - każde ciastko (obiekt) ma ten sam kształt, ale może mieć różne dodatki (atrybuty). Tworzenie obiektu = wyciśnięcie formy.

`__init__` - inicjalizacja obiektu

`__init__` to nie konstruktor! To inicjalizator - obiekt już istnieje, gdy `__init__` się uruchamia

```
class Czlowiek:
    def __init__(self, imie, nazwisko, wiek=20):
        self.imie = imie
        self.nazwisko = nazwisko
        self.wiek = wiek

c1 = Czlowiek('Jan', 'Kowalski')
print(c1.imie)      # Jan
print(c1.wiek)      # 20 (domyslne)
```

Co się dzieje pod maską?

1. `Czlowiek('Jan', 'Kowalski')`
2. Python wywołuje `__new__(cls)`
→ alokacja pamięci, nowy obiekt
3. Python wywołuje `__init__(self, ...)`
→ inicjalizacja atrybutów
4. Referencja zwracana do `c1`

⚠ `__init__` nie zwraca wartości (return None) - próba return z wartością to `TypeError`

self - referencja do bieżącego obiektu

```
class Klasa:
    def __init__(self):
        print('init', id(self)) # np. 26180694

k1 = Klasa()
print('obiekt:', id(k1))      # ten sam adres!

# self to ten sam obiekt co k1
# Python: k1.metoda() -> Klasa.metoda(k1)
```

Kluczowe fakty

Nazwa **self** to konwencja - można użyć dowolnej nazwy

W C++ odpowiednikiem jest **this**, ale tam jest niejawny

W Pythonie self jest jawny - zawsze pierwszy parametr metody instancji

```
# Dwa równoważne zapisy:
k1.szczekaj()           # Python zamienia na:
Pies.szczekaj(k1)       # jawne przekazanie obiektu

# Dlatego Pies.szczekaj() bez argumentu -> TypeError:
# szczekaj() missing 1 required positional argument: 'self'
```

Atrybuty instancji vs atrybuty klasy

Atrybut instancji (każdy obiekt ma swój)

```
class Czlowiek:
    def __init__(self, imie):
        self.imie = imie # instancji

c1 = Czlowiek('Jan')
c2 = Czlowiek('Anna')
print(c1.imie) # Jan
print(c2.imie) # Anna (rozne!)
```

Atrybut klasy (wspólny dla wszystkich)

```
class Czlowiek:
    gatunek = 'Homo sapiens' # klasy

    def __init__(self, imie):
        self.imie = imie

c1 = Czlowiek('Jan')
print(c1.gatunek) # Homo sapiens
print(Czlowiek.gatunek) # Homo sapiens
```

⚠ Pułapka z przypisaniem przez instancję

```
x = Czlowiek('Jan')
x.gatunek = 'Alien' # tworzy NOWY atrybut instancji!
print(x.gatunek) # Alien (przesłania atrybut klasy)
print(Czlowiek.gatunek) # Homo sapiens (klasa bez zmian)
print(type(x).gatunek) # Homo sapiens (dostęp do klasy)
```

Mutowalny atrybut klasy - pułapka!

✗ Źle — wspólna lista!

```
class Grupa:
    studenci = [] # atrybut klasy!

    def dodaj(self, s):
        self.studenci.append(s)

g1 = Grupa()
g2 = Grupa()
g1.dodaj('Jan')
print(g2.studenci) # ['Jan'] !!!
```

✓ Dobrze - lista w __init__

```
class Grupa:
    def __init__(self):
        self.studenci = [] # instancji!

    def dodaj(self, s):
        self.studenci.append(s)

g1 = Grupa()
g2 = Grupa()
g1.dodaj('Jan')
print(g2.studenci) # [] OK!
```

Mutowalne obiekty (list, dict, set) jako atrybuty klasy są współdzielone przez wszystkie instancje!

Dynamiczne atrybuty i inspekcja

setattr, getattr, hasattr, delattr, __dict__

```
c1 = Czlowiek('Jan', 'Kowalski')

setattr(c1, 'wiek', 34)
print(c1.wiek)          # 34

print(hasattr(c1, 'imie')) # True
print(hasattr(c1, 'pesel')) # False

val = getattr(c1, 'x', None) # None

delattr(c1, 'wiek')
print(hasattr(c1, 'wiek'))  # False
```

__dict__ - słownik atrybutów

```
c1 = Czlowiek('Jan', 'Kowalski')

print(c1.__dict__)
# {'imie': 'Jan', 'nazwisko': 'Kowalski',
#  'wiek': 20}

print(Czlowiek.__dict__.keys())
# ['__module__', '__init__',
#  'gatunek', '__dict__', ...]
```

Python pozwala dodawać atrybuty dynamicznie - ale to nie znaczy, że trzeba! W praktyce definiuj atrybuty w `__init__`.

Metody

instancji, @classmethod, @staticmethod

Metody instancji

Pierwszy parametr: self - dostęp do atrybutów i innych metod obiektu

```
class Konto:
    def __init__(self, wlasciciel, saldo=0):
        self.wlasciciel = wlasciciel
        self.saldo = saldo
        self.historia = []

    def wplata(self, kwota):          # metoda instancji
        self.saldo += kwota
        self.historia.append(f'+{kwota}')
        return self                  # fluent interface (opcjonalnie)

    def wypлата(self, kwota):
        if kwota > self.saldo:
            raise ValueError('Brak srodkow')
        self.saldo -= kwota
        self.historia.append(f'-{kwota}')

k = Konto('Jan', 1000)
k.wplata(500).wplata(200)    # chainowanie dzieki return self
print(k.saldo)               # 1700
```

@classmethod - alternatywne konstruktory

Pierwszy parametr: cls - referencja do klasy, nie do instancji

```
class Student:
    def __init__(self, first_name, last_name):
        self.first_name = first_name
        self.last_name = last_name

    @classmethod
    def from_string(cls, name_str):
        first, last = name_str.split(' ')
        return cls(first, last)    # cls, nie Student!

    @classmethod
    def from_json(cls, json_obj):
        return cls(json_obj['first'], json_obj['last'])

jan = Student.from_string('Jan Nowak')
print(jan.first_name)  # Jan
```

Użycie cls zamiast nazwy klasy sprawia, że @classmethod działa poprawnie z dziedziczeniem!

@staticmethod - funkcja „w klasie”

Brak dostępu do self ani cls - zwykła funkcja umieszczona w przestrzeni nazw klasy

```
class Student:
    @staticmethod
    def is_full_name(name_str):
        return len(name_str.split()) > 1

Student.is_full_name('Jan Nowak') # True
Student.is_full_name('Nowak')    # False

# Można też na instancji (ale bez sensu)
s = Student('Jan', 'Nowak')
s.is_full_name('Nowak')          # False
```

Kiedy użyć?

Metoda instancji

potrzebuje self (danych obiektu)

@classmethod

potrzebuje cls (klasa, fabryka)

@staticmethod

nie potrzebuje ani self, ani cls

```
# Kompletny przykład - 3 rodzaje metod
class MathHelper:
    pi = 3.14159 # atrybut klasy
    def pole_kola(self): return self.pi * self.r**2 # instancji (uzywa self)
    @classmethod
    def from_diameter(cls, d): return cls(d/2) # cls (tworzy obiekt)
    @staticmethod
    def dodaj(a, b): return a + b # statyczna (czysta funkcja)
```

__repr__ vs __str__

__repr__ — dla dewelopera

Jednoznaczna, techniczna.

Ideał: `eval(repr(obj)) == obj`

Wywoływany przez: `repr(obj)`,
interpreter, debugger, logi

__str__ - dla użytkownika

Czytelna, ładna.

Wywoływany przez: `str(obj)`,
`print(obj)`, f-stringi

Jeśli brak `__str__`, użyje `__repr__`

```
class Czlowiek:
    def __init__(self, imie, nazwisko, wiek=20):
        self.imie, self.nazwisko, self.wiek = imie, nazwisko, wiek

    def __repr__(self):
        return f'Czlowiek(imie={self.imie}, nazwisko={self.nazwisko}, wiek={self.wiek})'

    def __str__(self):
        return f'{self.imie} {self.nazwisko} (lat {self.wiek})'

c = Czlowiek('Jan', 'Kowalski')
print(repr(c)) # Czlowiek(imie=Jan, nazwisko=Kowalski, wiek=20)
print(str(c))  # Jan Kowalski (lat 20)
print(c)       # Jan Kowalski (lat 20)  <- print uzywa __str__
```

Properties i enkapsulacja

@property, konwencja `_`, `__`, `__slots__`

@property - atrybut z logiką

Metoda, która zachowuje się jak atrybut - bez zmiany API

```
class Okrag:
    def __init__(self, promien):
        self._promien = promien    # konwencja: _ = wewnetrzny

    @property
    def promien(self):              # getter
        return self._promien

    @promien.setter
    def promien(self, value):      # setter
        if value < 0:
            raise ValueError('Promien musi byc >= 0')
        self._promien = value

    @property
    def pole(self):                # read-only property
        return 3.14159 * self._promien ** 2

o = Okrag(5)
print(o.promien)    # 5          <- wyglada jak atrybut
print(o.pole)       # 78.54     <- wyliczane w locie
o.promien = 10      # OK, przechodzi walidacje
o.promien = -1      # ValueError!
```


Property - rekurencja bez podkreślnika

✗ Nieskończona rekurencja!

```
class Zle:
    def __init__(self, x):
        self.wartosc = x    # <- setter!

    @property
    def wartosc(self):
        return self.wartosc    # REKURENCJA

    @wartosc.setter
    def wartosc(self, v):
        self.wartosc = v    # REKURENCJA
```

✓ Użyj `_wartosc` (backing field)

```
class Dobrze:
    def __init__(self, x):
        self.wartosc = x

    @property
    def wartosc(self):
        return self._wartosc    # OK

    @wartosc.setter
    def wartosc(self, v):
        self._wartosc = v    # OK
```

Nazwa property (`wartosc`) i backing field (`_wartosc`) muszą się różnić! Inaczej setter wywoła sam siebie.

Konwencja `_` i `__` (name mangling)

Python nie ma true private — tylko konwencje i mechanizmy utrudniające dostęp

publiczny

```
self.imie = 'Jan'  
print(obj.imie) # OK
```

Bez podkreślnika.
Dostępny z zewnątrz.

`_`chroniony

```
self._saldo = 100  
print(obj._saldo) # OK!
```

Jedna `_`.
Konwencja: „nie dotykaj”.
Ale technicznie dostępny.

`__`prywatny

```
self.__pin = 1234  
print(obj.__pin) # Error!
```

Dwa `__`.
Name mangling:
`_Klasa__pin`

Name mangling: `__pin` → `_Klasa__pin` - utrudnia, ale nie blokuje dostępu!

__slots__ - optymalizacja pamięci

Blokuje __dict__, oszczędza RAM, zapobiega literówkom w nazwach atrybutów

```
class Punkt:
    __slots__ = ('x', 'y')

    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Punkt(1, 2)
p.x = 10      # OK
p.z = 30      # AttributeError!
```

Korzyści:

Mniej pamięci

brak __dict__ per instancja

Szybszy dostęp do atrybutów

stały offset zamiast hash lookup

Ochrona przed literówkami

p.z = 30 -> AttributeError

__slots__ jest często używany w klasach z milionami instancji (np. wiersze danych, grafy).

Dziedziczenie

`super()`, `override`, `isinstance()`, klasa `object`

Dziedziczenie - podstawy

Klasa potomna przejmuje atrybuty i metody klasy bazowej

```
class Zwierze:
    def __init__(self, imie, gatunek):
        self.imie = imie
        self.gatunek = gatunek

    def __str__(self):
        return f'{self.imie} ({self.gatunek})'

    def odglos(self):
        return '...'

class Pies(Zwierze):
    # dziedziczy z Zwierze
    def __init__(self, imie, rasa):
        super().__init__(imie, 'pies') # wywołanie __init__ rodzica
        self.rasa = rasa

    def odglos(self):
        # nadpisanie (override)
        return 'Hau!'

class Kot(Zwierze):
    def __init__(self, imie):
        super().__init__(imie, 'kot')

    def odglos(self):
        return 'Miau!'

p = Pies('Burek', 'owczarek')
print(p)          # Burek (pies)
print(p.odglos()) # Hau!
```

super() - wywołanie metody rodzica

Rozszerzanie zamiast nadpisywania

```
class Pracownik(Czlowiek):  
    def __init__(self, imie, nazwisko,  
                  firma, stanowisko):  
        super().__init__(imie, nazwisko)  
        self.firma = firma  
        self.stanowisko = stanowisko  
  
    def __str__(self):  
        base = super().__str__()  
        return f'{base}, {self.stanowisko}'
```

super() bez argumentów (Python 3)

super()

→ równoważne `super(Klasa, self)`

Zwraca obiekt proxy, który deleguje wywołania metod do klasy bazowej według kolejności MRO.

W Pythonie 2 trzeba było pisać:
`super(Klasa, self).__init__()`

⚠ `super()` używa MRO (Method Resolution Order), nie prostej hierarchii. Przy wielokrotnym dziedziczeniu kolejność może być zaskakująca - szczegóły na następnym wykładzie (OOP II).

isinstance() i isinstance()

isinstance(obj, klasa)

```
p = Pies('Burek', 'owczarek')

isinstance(p, Pies)      # True
isinstance(p, Zwierze)  # True (!
# respektuje dziedziczenie

isinstance(p, Kot)      # False
isinstance(p, (Pies, Kot)) # True
# krotka = OR

# UNIKAJ type() do sprawdzania typu!
type(p) == Pies         # True
type(p) == Zwierze     # FALSE (ignoruje!
```

issubclass(klasa, bazowa)

```
issubclass(Pies, Zwierze) # True
issubclass(Kot, Zwierze)  # True
issubclass(Pies, Kot)     # False
issubclass(Pies, object)  # True!

# Kazda klasa dziedziczy z object
issubclass(int, object)   # True
issubclass(str, object)   # True
issubclass(list, object)  # True
```

isinstance() > type() - bo respektuje dziedziczenie. Pisać: isinstance(x, int), nie: type(x) == int

Zapomnienie `super().__init__()`

✗ Brak `super().__init__()`

```
class Pracownik(Czlowiek):
    def __init__(self, imie, firma):
        # zapomnialam super()!
        self.firma = firma
```

```
p = Pracownik('Jan', 'ACME')
print(p.imie) # AttributeError!
# bo __init__ Czlowiek nigdy
# nie ustawil self.imie
```

✓ Zawsze wołaj `super().__init__`

```
class Pracownik(Czlowiek):
    def __init__(self, imie, firma):
        super().__init__(imie, 'N/A')
        self.firma = firma
```

```
p = Pracownik('Jan', 'ACME')
print(p.imie) # Jan - OK!
print(p.firma) # ACME
```

Jeśli klasa potomna definiuje `__init__`, MUSI wywołać `super().__init__()` - inaczej atrybuty rodzica nie zostaną zainicjalizowane.

__new__ vs __init__ - tworzenie obiektu

__new__ alokuje pamięć, __init__ ustawia stan — zazwyczaj wystarczy __init__

```
class Foo:
    def __new__(cls, *args, **kwargs):
        print(f'__new__: tworze {cls.__name__}')
        obj = super().__new__(cls)      # alokacja pamieci!
        return obj                     # MUSI zwrocic obiekt

    def __init__(self, a, b):
        print(f'__init__: inicjalizuje {a}, {b}')
        self.a, self.b = a, b

f = Foo(1, 2)
# __new__: tworze Foo
# __init__: inicjalizuje 1, 2

# Jesli __new__ NIE zwroci instancji cls -> __init__ NIE zostanie wywolany!
```

W 99% przypadków używasz tylko __init__. __new__ przydaje się przy Singletonie, niemutowalnych typach i metaklasach.

Przeciążanie operatorów

dunder methods — magia pod maską Pythona

Dunder methods - przegląd

Każdy operator i wiele wbudowanych funkcji mają odpowiadającą metodę specjalną

Tworzenie	<code>__new__</code> , <code>__init__</code> , <code>__del__</code>
Repr.	<code>__repr__</code> , <code>__str__</code> , <code>__format__</code>
Arytmetyka	<code>__add__</code> , <code>__sub__</code> , <code>__mul__</code> , <code>__truediv__</code> , <code>__floordiv__</code> , <code>__mod__</code> , <code>__pow__</code>
Odwrotne	<code>__radd__</code> , <code>__rsub__</code> , <code>__rmul__</code> , ...
In-place	<code>__iadd__</code> , <code>__isub__</code> , <code>__imul__</code> , ...
Porównania	<code>__eq__</code> , <code>__ne__</code> , <code>__lt__</code> , <code>__le__</code> , <code>__gt__</code> , <code>__ge__</code>
Kolekcje	<code>__len__</code> , <code>__getitem__</code> , <code>__setitem__</code> , <code>__delitem__</code> , <code>__contains__</code>
Wywołanie	<code>__call__</code>
Kontekst	<code>__enter__</code> , <code>__exit__</code>
Hash/Bool	<code>__hash__</code> , <code>__bool__</code>
Iteracja	<code>__iter__</code> , <code>__next__</code>
Rzutowanie	<code>__int__</code> , <code>__float__</code> , <code>__complex__</code> , <code>__index__</code>

Zasada: `func(obj) → obj.__func__()` | `obj1 + obj2 → obj1.__add__(obj2)`

__add__ i __sub__ - arytmetyka

```

class Liczba:
    def __init__(self, num):
        self.num = num

    def __repr__(self):
        return f'Liczba({self.num})'

    def __add__(self, other):
        if isinstance(other, Liczba):
            return Liczba(self.num + other.num)
        return Liczba(self.num + other)    # int/float

    def __radd__(self, other):              # 5 + Liczba(3)
        return self.__add__(other)

    def __iadd__(self, other):              # Liczba(3) += 5
        self.num += other.num if isinstance(other, Liczba) else other
        return self                        # MUSI zwrocic self!

k1, k2 = Liczba(3), Liczba(4)
print(k1 + k2)    # Liczba(7)
print(k1 + 5)     # Liczba(8)
print(5 + k1)     # Liczba(8)    <- __radd__
k1 += 10
print(k1)         # Liczba(13)   <- __iadd__

```

`__eq__`, `__lt__` - porównania

Mapowanie operatorów

```
x == y   → x.__eq__(y)
x != y   → x.__ne__(y)
x < y    → x.__lt__(y)
x <= y   → x.__le__(y)
x > y    → x.__gt__(y)
x >= y   → x.__ge__(y)
```

Mogą zwracać co chcą, ale
Python wywoła `bool()` na wyniku.

Przykład

```
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

    def __eq__(self, other):
        if not isinstance(other, Student):
            return NotImplemented
        return self.grade == other.grade

    def __lt__(self, other):
        if not isinstance(other, Student):
            return NotImplemented
        return self.grade < other.grade
```

Zwracaj `NotImplemented` (nie `NotImplementedError`!) gdy typ `other` jest nieobsługiwany - Python spróbuje odwrotnej metody.

`__len__`, `__getitem__` - interfejs kolekcji

```
class Zamowienie:
    def __init__(self, koszyk, klient):
        self.koszyk = list(koszyk)
        self.klient = klient

    def __len__(self):
        return len(self.koszyk)

    def __getitem__(self, key):
        return self.koszyk[key]          # obsluje tez slicing!

    def __contains__(self, item):
        return item in self.koszyk

    def __add__(self, item):
        nowy = self.koszyk.copy()
        nowy.append(item)
        return Zamowienie(nowy, self.klient)

z = Zamowienie(['rower', 'pilka', 'arbuz'], 'Jan')
print(len(z))          # 3
print(z[0])            # rower
print(z[1::-1])        # ['pilka', 'rower'] <- slicing dziala!
print('pilka' in z)    # True <- __contains__
z2 = z + 'but'
print(z2.koszyk)       # ['rower', 'pilka', 'arbuz', 'but']
```

__bool__, __hash__, __call__

__bool__

```
class Koszyk:
    def __init__(self):
        self.items = []

    def __bool__(self):
        return len(self.items)>0

k = Koszyk()
if not k: # False
    print('Pusty!')
```

__hash__

```
class Punkt:
    def __init__(self,x,y):
        self.x, self.y = x, y

    def __eq__(self,o):
        return (self.x,self.y)\
            == (o.x,o.y)

    def __hash__(self):
        return hash(
            (self.x,self.y))
```

__call__

```
class Mnozник:
    def __init__(self, n):
        self.n = n

    def __call__(self, x):
        return self.n * x

trzy = Mnozник(3)
print(trzy(10)) # 30
# obiekt jak funkcja!
print(callable(trzy))#T
```

⚠ Jeśli zdefiniujesz `__eq__`, Python automatycznie ustawia `__hash__ = None` → obiekt staje się niehashable!
Chcesz użyć w `set/dict`? Zdefiniuj też `__hash__`.

functools.total_ordering

Zdefiniuj `__eq__` i jedną metodę porównawczą - reszta wygeneruje się automatycznie

```
from functools import total_ordering

@total_ordering
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

    def __eq__(self, other):
        return self.grade == other.grade

    def __lt__(self, other):
        return self.grade < other.grade
```

Automatycznie dostępne:

```
a = Student('Jan', 85)
b = Student('Anna', 92)

a < b      # True   (zdefiniowane)
a <= b     # True   (wygenerowane!)
a > b      # False  (wygenerowane!)
a >= b     # False  (wygenerowane!)
a == b     # False  (zdefiniowane)
a != b     # True   (z __eq__)

sorted([b, a]) # [Jan(85), Anna(92)]
```

Zamiast pisać 6 metod porównawczych - wystarczy `__eq__` + jedna z: `__lt__`, `__le__`, `__gt__`, `__ge__`

Klasa Wektor2D - cz. 1: struktura

```
import math

class Wektor2D:
    def __init__(self, x=0.0, y=0.0):
        self.x = float(x)
        self.y = float(y)

    def __repr__(self):
        return f'Wektor2D({self.x}, {self.y})'

    def __str__(self):
        return f'({self.x}, {self.y})'

    def __abs__(self):
        # abs(v) -> dlugosc
        return math.hypot(self.x, self.y)

    def __bool__(self):
        # bool(v) -> True jesli != (0,0)
        return bool(abs(self))

    def __eq__(self, other):
        return (self.x, self.y) == (other.x, other.y)

    def __hash__(self):
        return hash((self.x, self.y))
```

Klasa Wektor2D - cz. 2: operatory

```
def __add__(self, other):
    return Wektor2D(self.x + other.x,
                    self.y + other.y)

def __mul__(self, scalar):          # v * 3
    return Wektor2D(self.x * scalar,
                    self.y * scalar)

def __rmul__(self, scalar):         # 3 * v
    return self * scalar

def __neg__(self):                  # -v
    return Wektor2D(-self.x, -self.y)

def __sub__(self, other):           # v1 - v2
    return self + (-other)

@property
def angle(self):
    return math.atan2(self.y, self.x)
```

Użycie:

```
v1 = Wektor2D(3, 4)
v2 = Wektor2D(1, 2)

print(v1 + v2) # (4.0, 6.0)
print(v1 * 3)  # (9.0, 12.0)
print(3 * v1)  # (9.0, 12.0)
print(-v1)     # (-3.0, -4.0)
print(abs(v1)) # 5.0
print(bool(v1)) # True

v0 = Wektor2D()
print(bool(v0)) # False

print(v1.angle) # 0.927 rad
```

To jest wzorcowa klasa - `__repr__`, `__str__`, arytmetyka, porównania, hash, bool, property. Polecam jako szablon!

Przykład z laboratorium - klasa Zespólona

```
import math

class Zespólona:
    def __init__(self, r=0.0, i=0.0):
        self.r, self.i = float(r), float(i)

    def conjugate(self):
        return Zespólona(self.r, -self.i)

    def argz(self):
        return math.atan2(self.i, self.r)

    def __repr__(self):
        return f'Zespólona({self.r}, {self.i})'

    def __str__(self):
        znak = '+' if self.i >= 0 else '-'
        return f'{self.r} {znak} {abs(self.i)}i'

    def __add__(self, other):
        if not isinstance(other, Zespólona): return NotImplemented
        return Zespólona(self.r + other.r, self.i + other.i)

    def __eq__(self, other):
        if not isinstance(other, Zespólona): return NotImplemented
        return self.r == other.r and self.i == other.i

a = Zespólona(1, 2)
b = Zespólona(3, -1)
print(a + b)  # 4.0 + 1.0i
```

Wszystko jest obiektem - nawet int!

Można wywołać dunder methods nawet na stałych liczbowych

```
# Operator + to w istocie __add__
(3).__add__(4)      # 7
'abc'.__len__()     # 3
[1,2,3].__getitem__(0) # 1
```

```
# UWAGA na pulapke parsera:
3.__add__(4)        # SyntaxError!
# Python mysli ze 3. to float!
```

```
# Sprawdź jakie metody ma int:
print(dir(42)) # [..., '__abs__', '__add__', '__and__', '__bool__',
# '__ceil__', '__class__', '__divmod__', '__eq__', '__float__',
# '__floor__', '__floordiv__', '__ge__', '__gt__', '__hash__',
# '__index__', '__int__', '__le__', '__lt__', '__mod__', ...]

# Porównaj z dir(object) - Twoja klasa dostaje część z nich za darmo!
```

Rozwiązania:

```
(3).__add__(4) # nawiasy
3 .__add__(4)  # spacja
int.__add__(3,4) # jawnie
```

Operator @ - mnożenie macierzy

`__matmul__` (od Python 3.5) - nazwa od mATrices, używany np. w NumPy

Własna implementacja:

```
class Wektor:
    def __init__(self, *args):
        self.data = list(args)

    def __matmul__(self, other):
        # iloczyn skalarny
        return sum(a*b for a,b
                    in zip(self.data,
                           other.data))

v1 = Wektor(1, 2, 3)
v2 = Wektor(4, 5, 6)
print(v1 @ v2) # 32
```

NumPy (naturalny kontekst):

```
import numpy as np

A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])

# Mnozenie macierzy:
C = A @ B
# array([[19, 22],
#        [43, 50]])

# Rownowazne:
np.matmul(A, B)
np.dot(A, B) # ale @ ladniejsze!
```

Operator @ jest też używany jako składnia dekoratorów — ale to zupełnie inny kontekst!

Singleton - `__new__` w praktyce

Wzorzec projektowy: tylko jedna instancja klasy w całym programie

```
class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = \
                super().__new__(cls)
        return cls._instance

    def __init__(self, value):
        self.value = value

s1 = Singleton('first')
s2 = Singleton('second')
print(s1 is s2)      # True!
print(s1.value)      # second
```

Kiedy użyć Singleтона?

Połączenie z bazą danych

jedno połączenie dla całej aplikacji

Logger

jeden logger, wspólna konfiguracja

Konfiguracja

jeden obiekt ustawień globalnych

⚠ Uwaga: `__init__` wywołuje się za każdym razem! Warto to obsłużyć.

Python nie ma przeciążania metod... ale...

W C++ można mieć dwie metody o tej samej nazwie ale różnych typach. W Pythonie — nie.

Problem:

```
class A:
    def metoda(self, x):
        print('int:', x)

    def metoda(self, x, y): # nadpisuje!
        print('two:', x, y)

a = A()
a.metoda(1)      # TypeError!
a.metoda(1, 2)   # two: 1 2
```

Rozwiązanie z multipledispatch:

```
from multipledispatch import dispatch

class A:
    @dispatch(int)
    def metoda(self, x):
        print('int:', x)

    @dispatch(int, int)
    def metoda(self, x, y):
        print('two:', x, y)

a = A()
a.metoda(1)      # int: 1
a.metoda(1, 2)   # two: 1 2
```

Python 3.4+ ma też `@functools.singledispatch` dla funkcji (nie metod). `multipledispatch` to osobna paczka (pip install).

Dobre praktyki OOP

Zawsze definiuj `__repr__`

Bez niego debugowanie to koszmar: `<Foo object at 0x...>`

Atrybuty w `__init__`

Nigdy nie polegaj na dynamicznym dodawaniu atrybutów później

Używaj `isinstance`, nie `type`

`isinstance(x, int)` respektuje dziedziczenie

`__eq__` wymusza `__hash__`

Jeśli zdefiniujesz `__eq__`, zdefiniuj też `__hash__` (albo ustaw `= None`)

Preferuj kompozycję

„Ma” (has-a) często lepsze niż „jest” (is-a) — nie dziedzicz na siłę

Wspólny interfejs

Dunder methods dają naturalny, „pythoniczny” interfejs klasy

Walidacja w setterze

`@property` + `setter` = bezpieczne dane bez zmiany API

Dokumentuj klasę

`docstring: """Krótki opis.\n\nAtrybuty: ..."""`

Co wypisze ten kod?

```
class A:
    x = []
    def add(self, val):
        self.x.append(val)

class B(A):
    pass

a = A()
b = B()
a.add(1)
b.add(2)

print(a.x)    # ???
print(b.x)    # ???
print(A.x)    # ???
print(a.x is b.x) # ???
```

Odpowiedź:

[1, 2]

[1, 2]

[1, 2]

True

x = [] jest atrybutem klasy A.
Klasa B dziedziczy go.
Obie instancje modyfikują
tę samą listę!

Rozwiązanie: przenieś x
do __init__ jako self.x = []

Programowanie obiektowe I

1 `class + __init__ + self`

fundamenty tworzenia klas

2 **Atrybuty instancji vs klasy**

uważaj na mutowalne!

3 `@classmethod`, `@staticmethod`

`@property` — kontrola dostępu

4 `__repr__` (debug) vs `__str__` (print)

zawsze definiuj `__repr__`

5 **Enkapsulacja: `_` i `__`**

name mangling, `__slots__`

6 **Dziedziczenie + `super()`**

zawsze wołaj `super().__init__()`

7 `isinstance()` > `type()`

respektuje dziedziczenie

8 **Dunder methods**

naturalny interfejs: `+`, `==`, `len()`, `[]`

9 **NotImplemented**

nie `NotImplementedError`!

10 `__eq__` kasuje `__hash__`

pamiętaj o hashowaniu